Apply filters to SQL queries

Project description

This project focuses on crafting the right SQL SELECT queries that fit the teams need.

Retrieve after hours failed login attempts

My team is investigating failed login attempts that were made after business hours. I need to retrieve this information from the login activity.

```
SELECT * FROM log_in_attempts WHERE login_time > ‘18:00:00’ AND
success = 0;
```

This query returns all login attempts with all the columns and the records that happened after 6 PM (after hours), using login_time column and the greater than operator in the filtering condition.

Retrieve login attempts on specific dates

My team is investigating a suspicious event that occurred on '2022-05-09'. I need to retrieve all login attempts that occurred on this day and the day before ('2022-05-08').

```
SELECT * FROM log_in_attempts WHERE login_date = ‘2022-05-08’ OR
login_date = ‘2022-05-09’;
```

This query returns all login attempts with all the columns for the records that happened on the dates 2022-05-08 and 2022-05-09, using the OR operator to join the 2 dates.

Retrieve login attempts outside of Mexico

My team is investigating logins that did not originate in Mexico.
Note: the country field includes entries with 'MEX' and 'MEXICO'.

```
SELECT * FROM log_in_attempts WHERE NOT country LIKE ‘MEX%’;
```

This query returns all login attempts with all the columns for the records that happened outside of Mexico, using the NOT operator and the LIKE operator to ensure solid results with inconsistent data entry.

Retrieve employees in Marketing

My team is updating employee machines. I need to obtain the information about employees in the 'Marketing' department who are located in all offices in the East building (such as 'East-170' or 'East-320').

```
SELECT * FROM employees WHERE department = ‘Marketing’ AND
office LIKE ‘EAST%’;
```

In this query we used the AND operator to retrieve all the records that match the two conditions together, also we used the LIKE operator to get all the employees that are working exclusively in the east building.

Retrieve employees in Finance or Sales

My team needs to perform a different update to the computers of all employees in the Finance or the Sales department.

```
SELECT * FROM employees WHERE department = ‘Finance’ OR
department = ‘Sales’;
```

Here we used OR operator to fit the request. The OR operator retrieves records that archive one of the conditions.

Retrieve all employees not in IT

My team needs to make one more update. This update was already made to employee computers in the Information Technology department. The team needs information about employees who are not in that department.

```
SELECT * FROM employees WHERE NOT department = ‘Information
Technology’;
```

Here we used the NOT operator to reverse the condition. This query retrieves all the records that do not match the condition.

Summary

In this project I conducted simple SQL SELECT queries To retrieve relevant information to the task at hand. First I crafted a query to help with investigating a potential security incidents. Then I retrieved information relevant to managing the team.